word2vec

Peter N. Robinson

The Jackson Laboratory for Genomic Medicine

July 2, 2018

Outline

1. Introduction
2. n-gram and skip-gram models
3. word2vec

word2vec

The purpose of this presentation is to provide an introduction to the topic of vector embeddings in the context of the word2vec algorithm

Main sources

- Mikolov T et al. (2013) Distributed Representations of Words and Phrases and their Compositionality. NIPS'13 Proceedings of the 26th International Conference on Neural Information Processing Systems—Volume 2 Pages 3111-3119
- https://www.tensorflow.org/tutorials/word2vec
- Stanford online course on deep learning and NLP (especially lecture #2)

Applications of word2vec

Distributed representations of words in a vector space help learning algorithms to achieve better performance in natural language processing tasks by **grouping similar words**.

- The word representations computed using neural networks are very interesting because the learned vectors explicitly encode many linguistic regularities and patterns.
- Many of these patterns can be represented as linear translations.
- For example, the result of a vector calculation `vec("Madrid") - vec("Spain") + vec("France")` is closer to `vec("Paris")` than to any other word vector

Outline

1 Introduction

2 n-gram and skip-gram models

3 word2vec

skip-gram model

n-gram model

In the fields of computational linguistics, an **n-gram** is a contiguous sequence of n words (or other items) from a given sample of text or speech.

- One use case of n-grams is to predict the next word in a phrase using an $(n-1)$-order Markov model

n-gram model

e.g., from this phrase "to be or not to be", we can generate the following n-grams

- One-gram: to/be/or/not/to/be
- Two-gram to be/be or/or not/not to/to be
- Three-gram to be or/be or not/or not to/not to be
- ...

skip-gram model

skip-grams are generalizations of n-grams in which we are allowed to skip words, with the components occuring at not more than a specified distance to each other

skip-gram

an n-gram is a consecutive subsequence of length n of some sequence of tokens $w_1, \ldots, w_n$. A k-skip-n-gram is a length-n subsequence where the components occur at distance at most k from each other.

For instance, some 1-skip 4-grams of "to be or not to be" are:

- to be or not
- to or not be
- be or to be

skip-gram model

The training objective of the Skip-gram model is to find word representations that are useful for predicting the surrounding words in a sentence or a document.

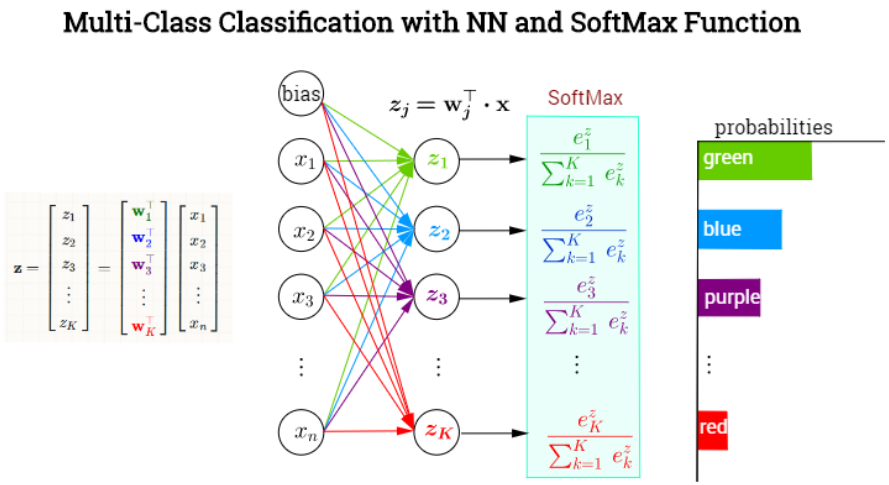
Given a sequence of training words $w_1, w_2, \ldots, w_T$, we maximize as follows

$$\max \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-c \leq j \leq c \\ c \neq j}} \log p(w_{t+j} | w_t) \tag{1}$$

In this formulation of the skip gram model, all forwards and backwards skips of up to $c - 1$ words are considered. Larger c results in more training examples and thus can lead to a higher accuracy, at the expense of the training time.

skip-gram model

The basic Skip-gram formulation defines $p(w_t + j|w_t)$ using the softmax function, which is also often used for multiclass predictors in neural networks



- softmax turns a collection of numbers into a probability distribution ($z_i \rightarrow \text{softmax}(z_i) = p_i$ implies that $p_i \geq 0$ (because of the exponentiation) and $\sum p_i = 1$ (because of the normalization in the denominator)

skip-gram model

For the skip-gram model, we use $\mathbf{v}_w$ and $\mathbf{v}'_w$ as the "input" and "output" vector representations of w and W is the number of words in the vocabulary

$$p(w_O|w_I) = \frac{e^{({\mathbf{v}'_{w_O}}^T \mathbf{v}_{w_I})}}{\sum_{w=1}^{W} e^{({\mathbf{v}'_w}^T \mathbf{v}_w)}} \tag{2}$$

- This formulation is impractical because the cost of computing $\nabla \log p(w_O|w_I)$ is proportional to W, which is often large ($10^5 - 10^7$ terms)
- We will discuss what is meant by input and output vectors further on.

Shortcomings addressed by skip-gram model

Word representations are limited by their inability to represent idiomatic phrases that are not compositions of the individual words.

Boston Globe

For example, "Boston Globe" is a newspaper, and so it is not a natural combination of the meanings of "Boston" and "Globe"

- Therefore, using vectors to represent the whole phrases makes the Skip-gram model considerably more expressive.

Outline

1 Introduction

2 n-gram and skip-gram models

3 word2vec

word2vec

Word2vec is a group of related models that are used to produce word embeddings, based on a series extensions of the original Skip-gram model.

- shallow, two-layer neural networks that are trained to reconstruct linguistic contexts of words.
- Input: a large corpus of text
- Output: a vector space, typically of several hundred dimensions, with each unique word in the corpus being assigned a corresponding vector in the space.
- Word vectors are positioned in the vector space such that words that share common contexts in the corpus are located in close proximity to one another in the space.

word vectors

for some previous systems, each word is considered in isolation. And thus if we are looking at 10,000 words and represent each word as a vector, we get a vector with a single "1" entry and 9999 "0" entries

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & \dots & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{3}$$

These vectors are called one-hot vectors, and could be very long (e.g., 50,000 words or even 13 million for Google...). These vectors have **no natural notion of similarity**, e.g., the dot product between the vector for apple and pear would be zero.

distributional similarity

You shall know a word by the company it keeps J.R. Firth, 1957
Dime con quien andas, y te dirè quien eres

- Word2vec essentially looks at the context in which a word appears
- Find the distribution of a word's neighbors in a large corpus of text—this will represent the "meaning" of the word

Goal of word2vec

Choose a dense vector for each word that will be good at predicting which words it co-appears with

Define a model that predicts the context surrounding a center word w_t using word vectors

$$p(\text{context}|w_t) = \ldots \tag{4}$$

To train the model, we look at lots of positions in a big language corpus and adjust the vector representations of words to minimize a loss function.

- So how do we define the vectors and the loss function?
- What dimension should the vectors have ? (hopefully less than the total number of words ...)

word2vec

For each word w_t $t = 1, \ldots T$, predict surrounding words within a radius m

Loss function[1]

$$J(\Theta) = \prod_{t=1}^{T} \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} p(w_{t+j} | w_t, \Theta) \tag{5}$$

i.e., for each position t in the text, take a window of size $2m$ and look at the probabilities of the context words in this window. We will minimize the negative log likelihood (adding a normalizing constant $\frac{1}{T}$ to get the per word probabilities).

$$\ell(\Theta) = -\log J(\Theta) = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log p(w_{t+j} | w_t, \Theta) \tag{6}$$

[1] aka objective function, cost function

word2vec

The major parameters in this model are the vector representations of the words—all of which are represented here as Θ. There are several hyperparameters.

We now need to define the term $p(w_{t+j}|w_t, \Theta)$. We will start off with the **softmax** function mentioned above, where o means the outside word and c means the center word for some t and some j, i.e., w_{t+j} is shown here as o and w_t is shown as c.

$$p(w_{t+j}|w_t, \Theta) = p(o|c) = \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)} \tag{7}$$

Here, V is the total number of vocabulary of words (different from T the total number of words in a large corpus (where individual words are repeated).

word2vec

Each word actually has two vector representations

- $\mathbf{v}_c$ is the representation for a word in the center of the window
- $\mathbf{u}_o$ is the representation for a word in the context (outside) of the center word
- Note that in this model the position of the context word in the window is not important

This makes the math a little easier and works a little better in practice.

word2vec: initialize from one hot

Initializing the model from a one-hot vector for some word t gives us...

We have

$$\mathbf{w}_t \mathbf{W} = \mathbf{v}_c \tag{8}$$

where $\mathbf{w}_t$ is a $V \times 1$ vector, $\mathbf{W}$ is a randomly initialized $d \times V$ matrix, and $\mathbf{v}_c$ is a $d \times 1$ vector.

$$\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} \dots & \dots & 0.2 & \dots & \dots \\ \dots & \dots & -1.4 & \dots & \dots \\ \dots & \dots & 0.3 & \dots & \dots \\ & & \vdots & & \\ \dots & \dots & 0.7 & \dots & \dots \\ \dots & \dots & 0.6 & \dots & \dots \end{bmatrix} = \begin{bmatrix} 0.2 \\ -1.4 \\ 0.3 \\ \vdots \\ 0.7 \\ 0.6 \end{bmatrix} \tag{9}$$

word2vec: calculate the loss

Continuing the example from the previous page, and imagining that we are looking at one specific instance, where we have a vector called u_{truth}, we would calculate the loss as follows.

We have a second $(V \times d)$ matrix **U** that contains the context word vector representations (just as **W** contains the center word representations).

$$\mathbf{v}_c\mathbf{U} = \dots \tag{10}$$

as

$$\begin{bmatrix} 0.2 \\ -1.4 \\ 0.3 \\ \vdots \\ 0.7 \\ 0.6 \end{bmatrix} \begin{bmatrix} \dots & \dots & 0.4 & \dots & \dots \\ \dots & \dots & 0.4 & \dots & \dots \\ \dots & \dots & 1.3 & \dots & \dots \\ & & \vdots & & \\ \dots & \dots & -0.7 & \dots & \dots \\ \dots & \dots & 0.1 & \dots & \dots \end{bmatrix} = \begin{bmatrix} 0.3 \\ 0.01 \\ -0.2 \\ \vdots \\ 0.9 \\ 0.1 \end{bmatrix} \tag{11}$$

word2vec: calculate the loss (2)

To calculate the loss in this example, we take the softmax of the product of $\mathbf{v}_c$ and $\mathbf{U}$ and compare it to the truth vector (which is a one-hot vector containing the "truth", i.e., the word that really was at the indicated position of the context).

$$\text{softmax}(\mathbf{v}_c\mathbf{U}) = \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ \vdots \\ p_{V-1} \\ p_V \end{bmatrix} \tag{12}$$

If the truth vector has a one at position i, then we would feed $1 - p_i$ to our loss function.

word2vec: Train model

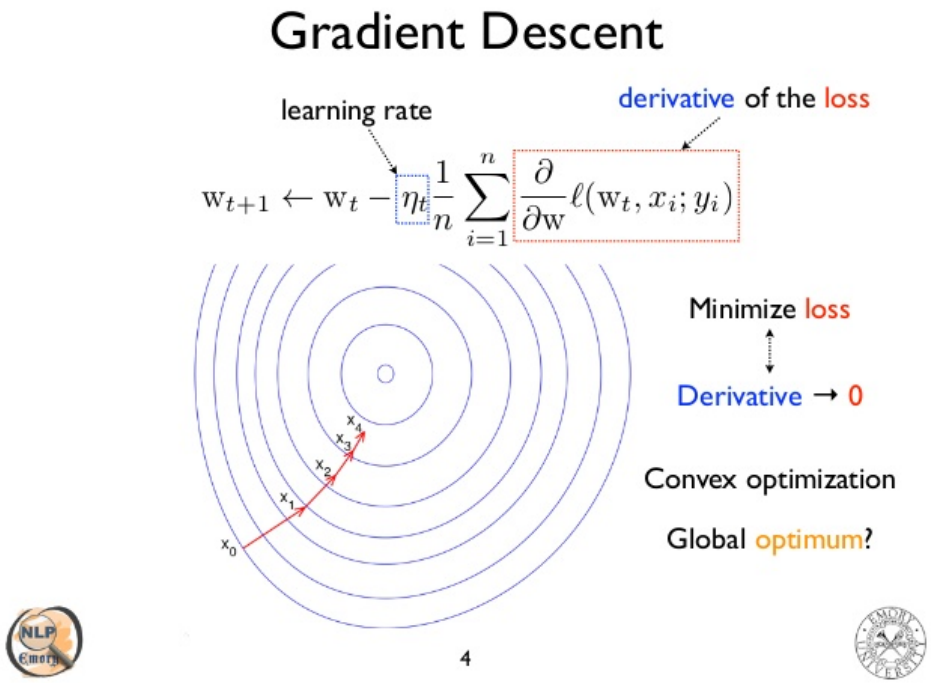
As is usual in deep learning, we will train the model using gradient descent. We first define a vector Θ with all parameters (both the center ($\mathbf{v}$) and the outside ($\mathbf{u}$) vectors for all words

$$\Theta = \begin{bmatrix} \mathbf{v}_{aardvark} \\ \vdots \\ \mathbf{v}_{zebra} \\ \mathbf{u}_{aardvark} \\ \vdots \\ \mathbf{u}_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV} \tag{13}$$

thus, there is a d dimensional vector for each word, and we have a total of V words, each of which is represented twice. All of this gets flattened into a $2dV \times 1$ dimensional vector we call Θ.

gradient descent

word2vec uses gradient descent to find "optimum" values of its parameters. Note that backpropagation as used in artificial neural networks/deep learning is basically a form of gradient descent with some extra bells and whistles such as memoization.



gradient descent

We use gradient descent to obtain a collection of vectors that minimize the cost.

The optimization equations for each word are as follows (using a center for for this example; it is analogous for context words).

$$
\begin{aligned}
\log p(w_{t+j}|w_t) &= \log p(o|c) \\
&= \log \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)} \\
&= \mathbf{u}_o^T \mathbf{v}_c - \log \sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)
\end{aligned}
$$

We will separate these into the two terms for clarity

gradient descent

For the first term we have:

$$\begin{aligned} \nabla \mathbf{u}_o^T \mathbf{v}_c &= \frac{\partial \mathbf{u}_o^T \mathbf{v}_c}{\partial \mathbf{v}_c} \\ &= \mathbf{u}_o \end{aligned}$$

gradient descent

For the second term recall the chain rule $D\{f(g(x))\} = f'(g(x)) \cdot g'(x)$. In the following we will ignore the sign of the second term for the moment.

$$\frac{\partial}{\partial \mathbf{v}_c} \log \sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c) \quad = \quad \frac{1}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)} \frac{\partial}{\partial \mathbf{v}_c} \sum_{x=1}^{V} \exp(\mathbf{u}_x^T \mathbf{v}_c)$$

Note that in this equation, log pays the part of the outer function f and $\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)$ is the inner function g; the first derivative of $\log x$ is $\frac{1}{x}$.

gradient descent

continuing

$$
\begin{aligned}
\frac{\partial}{\partial \mathbf{v}_c} \sum_{x=1}^{V} \exp(\mathbf{u}_x^T \mathbf{v}_c) &= \sum_{x=1}^{V} \frac{\partial}{\partial \mathbf{v}_c} \exp(\mathbf{u}_x^T \mathbf{v}_c) \\
&= \sum_{x=1}^{V} \exp(\mathbf{u}_x^T \mathbf{v}_c) \frac{\partial}{\partial \mathbf{v}_c} \mathbf{u}_x^T \mathbf{v}_c \\
&= \sum_{x=1}^{V} \exp(\mathbf{u}_x^T \mathbf{v}_c) \mathbf{u}_x
\end{aligned}
$$

where the second line follows from the chain rule and the fact that $\frac{de^x}{dx} = e^x$. Note we changed the summation variable of the inner sum (which does not change the value) to avoid "interference".

gradient descent

continuing and pulling all the parts together

$$\begin{aligned}\frac{\partial}{\partial \mathbf{v}_c} \log \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)} &= \mathbf{u}_o - \frac{1}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)} \sum_{x=1}^{V} \exp(\mathbf{u}_x^T \mathbf{v}_c)\mathbf{u}_x \\ &= \mathbf{u}_o - \sum_{x=1}^{V} \frac{\exp(\mathbf{u}_x^T \mathbf{v}_c)}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)} \mathbf{u}_x\end{aligned}$$

where the second line follows from the fact that $\frac{1}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)}$ is just a scalar number and can be pulled into the sum. We now recall that the definition of our softmax function for the probability that word x occurs as a context word of center word c is defined using the softmax function as

$$p(x|c) = \mathbf{u}_o - \sum_{x=1}^{V} \frac{\exp(\mathbf{u}_x^T \mathbf{v}_c)}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^T \mathbf{v}_c)} \mathbf{u}_x \tag{14}$$

gradient descent

Therefore, the gradient actually has the form of an observed value ($\mathbf{u}_o$) subtracted by the expectation across all context words

$$\frac{\partial}{\partial \mathbf{v}_c} \log p(w_{t+j}|w_t) = \mathbf{u}_o - \sum_{x=1}^{V} p(x|c)\mathbf{u}_x \tag{15}$$

Note that $p(x|c)$ is a vector with dimensions $V \times 1$.

Optimization

The gradients would be calculated for all $2Vd$ parameters in the model, i.e., all $\mathbf{v}_c$ and $\mathbf{u}_o$

gradient descent

To minimize the objective function $J(\Theta)$ over the entire batch of training data, we would perform an update step using a proper learning rate α

$$\Theta_j^{(i+1)} \leftarrow \Theta_j^{(i)} - \alpha \frac{\partial}{\partial \Theta_j} J(\Theta)$$

alternatively

$$\Theta_j^{(i+1)} \leftarrow \Theta_j^{(i)} - \alpha \nabla_{\Theta_j} J(\Theta)$$

We iterate, walking towards the minimum. The gradient points in the direction of the steepest descent towards the minimum, but needs to be small enough that we do not overshoot.

gradient descent

One of the issues that word2vec strives to overcome is the fact that we may have many millions or even billions of tokens to train over, and so one step of the gradient descent would take lots of time

A simple approach might be to use mini batch GD or stochastic GD, in which calculate the gradient based on only one example at each step. This is not guaranteed to find the correct gradient, but in practice it works very well, and the noise that it introduces into the deep learning algorithm tends to prevent overfitting.

word2vec parameters

What about the dimensionality parameter d?

The dimensionality of the two vectors that represent each word is set as a hyperparameter. Reportedly, values of ca. 100–1000 work well (one could test values at intervals of 100 to find the best value).

word2vec: scaling up

Neural probabilistic language models are traditionally trained using the maximum likelihood (ML) principle to maximize the probability of the next word (for "target") given the previous words (for "history") in terms of a softmax function,

$$P(w_t|h) = \text{softmax}(\text{score}(w_t, h)) \tag{16}$$

$$= \frac{\exp\{\text{score}(w_t, h)\}}{\sum_{\text{Word w' in Vocab}} \exp\{\text{score}(w', h)\}} \tag{17}$$

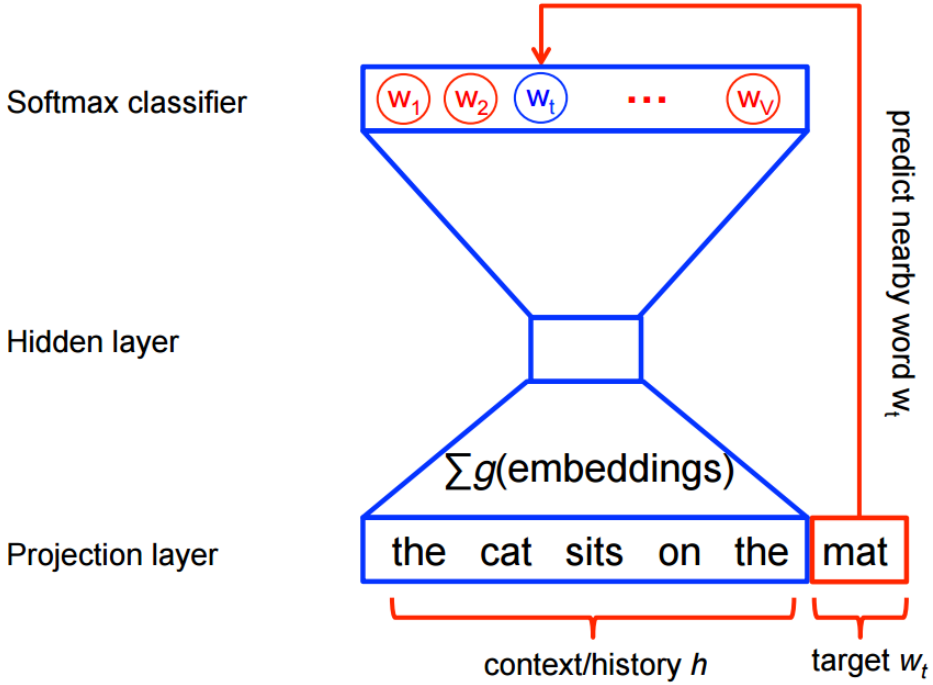
word2vec: scaling up

The model is trained by maximizing its log-likelihood on the training set, i.e. by maximizing

$$J_{\mathrm{ML}} = \log P(w_t|h) \tag{18}$$

$$= \mathrm{score}(w_t, h) - \log \left(\sum_{\text{Word w' in Vocab}} \exp\{\mathrm{score}(w', h)\} \right). \tag{19}$$

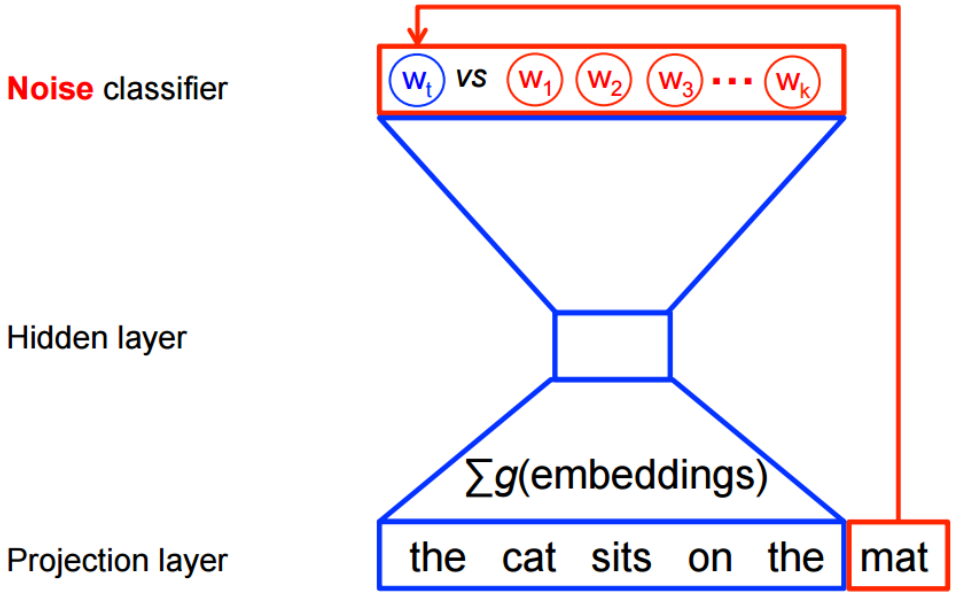
word2vec: scaling up



- this is very expensive, because we need to compute and normalize each probability using the score for all V other words w' in the current context h, at every training step.

word2vec: scaling up

word2vec therefore is trained to discriminate the real target words w_t from k imaginary (noise) words $\tilde{w}$, in the same context.



- Details are in the original publication and an intuitive explanation is available at the tensorflow tutorial on word2vec.

python implementation

Note that the directory where these slides live has a python3 script for performing word2vec that is taken from a referenced online tutorial. It is worth going over if you know python.